

Campus Log Anomaly Detection under Proxy Labels: Validity Limits of Archive Splits and Masked TCN–Transformers

Tien Thanh Cao^{1*}, Hoang Tu Trinh¹, and Manh Ha Tran¹

Faculty of Information Technology,
Ho Chi Minh City University of Foreign Languages – Information Technology,
Ho Chi Minh City, Vietnam
thanhct@huflit.edu.vn, 23dh113972@st.huflit.edu.vn, hatm@huflit.edu.vn

Abstract. Campus log anomaly studies often treat archive order as time and window OR-aggregation as a label. We measure how those choices distort conclusions on a selective HUFLIT stream (1,000,000 deduplicated lines; custom similarity parser; HTTP/keyword *proxy* labels). On a leaky stream-position split, test-window prevalence reaches 80.5% because a window is positive if any of $L=20$ lines is positive; an always-anomaly classifier then obtains $F1 \approx 0.892$, while a masked TCN–Transformer under $\tau = \mu + 3\sigma$ scores only 0.0395 ± 0.0163 F1 despite ROC-AUC 0.7226 ± 0.0178 (AUPRC lift over prevalence $\approx +0.031$). After global line-hash deduplication, source/member-bounded windows, and a day-held-out split (train 15–17 Jun, val 18 Jun, test 19 Jun 2026), test prevalence falls to 56.4% (always-anomaly F1 baseline 0.721), yet masked models collapse to near-chance ranking (TCN–Transformer ROC-AUC 0.5315 ± 0.112 ; no-TCN 0.4933 ± 0.009), while PCA retains a modest AUPRC lift (+0.098). Threshold sweeps show $\mu + 3\sigma$ suppresses recall; looser rules inflate F1 toward the high-prevalence trivial baseline. The TCN adds latency (batch-1 51.1 vs. 18.5 ms) without a consistent accuracy benefit. Local LLM RCA is left to future work.

Keywords: AIOps · Log Anomaly Detection · Proxy Labels · Experimental Validity · TCN–Transformer · Campus Infrastructure.

1 Introduction

Campus IT services emit heterogeneous operational logs. Sequence detectors can score unusual template patterns [2–5], but archive packaging, duplicate backups, and heuristic labels can invent “temporal” structure that models then appear to learn.

This paper treats those validity threats as first-class experimental objects. We study a masked TCN–Transformer and a no-TCN ablated Transformer on HUFLIT selective extracts, with two protocols: (A) a leaky stream-position split

* Corresponding author: thanhct@huflit.edu.vn

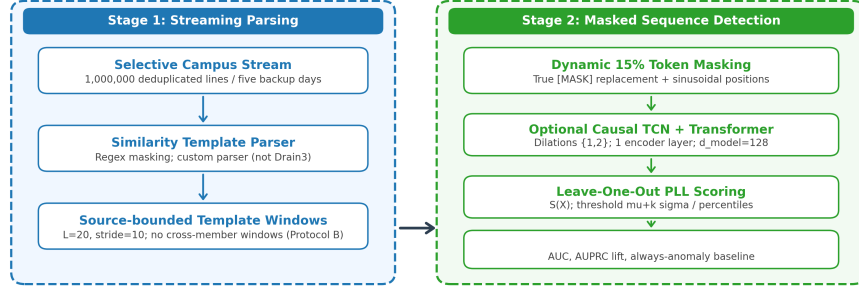


Fig. 1. Measured detector pipeline: streaming parsing and masked PLL scoring. Local LLM RCA is out of scope.

that mirrors a common but flawed archive ordering; (B) a deduplicated, source-bounded, day-held-out split. We add trivial baselines (always-anomaly; no-skill $\text{AUPRC} = \text{prevalence}$), report AUPRC lift over prevalence, and sweep validation thresholds. We do *not* claim an LLM root-cause system; local narration is deferred.

The contributions are:

- **Validity protocol:** quantify how OR window labels and archive-order splits inflate prevalence and inflate apparent F1 relative to an always-anomaly trivial baseline; retain dual-split evaluation artifacts (`huflit_baselines.json`, `huflit_v2.*`).
- **Dedup + day split:** global SHA-1 line dedup, windows that never cross (day, source, member), train/val/test by backup day.
- **Threshold-aware reading:** $\mu + k\sigma$ and percentile sweeps plus a validation-oracle diagnostic, separating ranking from calibration.
- **Negative TCN evidence:** on both protocols the TCN does not consistently beat a masked Transformer without TCN, while increasing leave-one-out scoring latency.

2 Related Work

2.1 Log Parsing and Template Extraction

Log parsing transforms free-form messages into event templates by separating constants from dynamic parameters [1, 12]. Drain uses a fixed-depth parse tree [1]; Spell uses streaming sequence matching [12]. Our artifact does *not* implement Drain or Drain3. It applies regex masking and a linear scan over equal-length templates, merging tokens when positional similarity exceeds 0.5. We call this the *custom similarity parser* and avoid attributing Drain3 properties or throughput to it.

2.2 Deep Learning for Log Anomaly Detection

Sequence-based anomaly detection models learn temporal transition dynamics over sequences of log templates [2, 3, 6]. Du et al. [2] introduced DeepLog (LSTM next-token prediction). Meng et al. [3] proposed LogAnomaly with semantic template vectors. SwissLog [5] and LogRobust [4] address instability and interleaved threads. Classical reconstruction baselines such as PCA on count vectors remain competitive on some structured logs [6]. Recent Transformer–TCN hybrids [19], multi-scale semantic models [16], and contrastive masked objectives [17] continue this line. Public corpora such as Loghub [11] remain the standard sanity setting; we defer a full HDFS/BGL re-run under the present harness to future work.

2.3 LLMs for Log Analytics

LLM-assisted detectors include KRONE [14], MaidLog [15], CoLA [20], and Smart Eye [18]. LLM4Log [13] surveys the area; cloud RCA recommenders are studied in [9, 10]. Our focus is experimental validity on campus archives rather than a new LLM narrator.

Table 1. Qualitative comparison of related systems. Latency is *not* cross-comparable: this work reports batch-1 leave-one-out PLL; DeepLog/LogAnomaly-lite legacy batched scoring is NR for batch-1.

Approach	Parser	Detector	RCA	Local LLM RCA	CPU ms
DeepLog [2]	Spell	LSTM next-token	Scalar score	×	NR [†]
LogAnomaly [3]	Template+FT	Semantic LSTM	Scalar score	×	NR ^{†,‡}
LogRobust [4]	Drain	Bi-LSTM+Attn	Scalar score	×	NR
SwissLog [5]	Online	Attentive Bi-LSTM	Heuristic loc.	×	NR
KRONE [14]	LLM hierarchy	Hybrid+LLM	Nested LLM	Cloud/API-oriented	NR
This work	Custom similarity	Masked TCN–Transformer	Deferred	×	51.1 [§]

[†]Legacy batched ms retained in JSON but not relabeled as batch-1 PLL. [‡]LogAnomaly-lite reimplement on our templates, not the authors’ FastText stack. [§]Batch-1 leave-one-out PLL on the deduplicated day-split (CPU, 2 threads).

3 Methodology

3.1 System Architecture Overview

The measured system has two stages: (i) **Parsing**: selected syslog messages are tokenized by the custom similarity parser; (ii) **Masked sequence scoring**: sliding template windows are corrupted by true token replacement, encoded by an optional causal TCN plus Transformer, and scored by leave-one-position-out pseudo-log-likelihood (PLL). Local LLM+RAG narration is *out of scope* (Future Work).

3.2 Masked TCN-Transformer Denoising Sequence Model

We formulate the detector as a masked template language model: 15% of input tokens are replaced by a reserved mask token, representations pass through a

Temporal Convolutional Network (TCN) [7] and Transformer self-attention [8], and a linear vocabulary head predicts the original identities only at selected positions.

Embedding and Dynamic Masking A sliding window of length $L = 20$ with stride $S = 10$ extracts template sequences $\mathbf{X} = [T_1, T_2, \dots, T_L]$. During training, 15% of tokens are replaced with [MASK] ($ID = |\mathcal{V}| + 1$), distinct from padding ($ID = 0$). Padding masks are applied in self-attention. Embeddings use sinusoidal positional encodings [8]:

$$\mathbf{E} = \text{Embedding}(\mathbf{X}) + \mathbf{P}_{\text{pos}} \in \mathbb{R}^{L \times d_{\text{model}}} \quad (1)$$

Temporal Convolutional Network (TCN) Layer Dilated causal convolutions [7] with $d \in \{1, 2\}$ extract local patterns without future leakage:

$$\mathbf{y}(t) = (\mathbf{x} *_d f)(t) = \sum_{i=0}^{k-1} f(i) \cdot \mathbf{x}(t - d \cdot i) \quad (2)$$

$$\mathbf{H}_{\text{TCN}} = \text{LayerNorm}(\text{Proj}(\mathbf{E}) + \text{GELU}(\text{CausalTCN}(\mathbf{E}))) \quad (3)$$

Transformer Encoder and Scoring A single Transformer encoder layer (pre-LN, multi-head attention, GELU FFN) contextualizes the sequence. Training minimizes masked cross-entropy. Inference uses leave-one-position-out PLL:

$$S(\mathbf{X}) = \frac{1}{L_{\text{valid}}} \sum_{t=1}^{L_{\text{valid}}} -\log P(T_t = t_t \mid \mathbf{X}_{\setminus \{t\}}) \quad (4)$$

The primary threshold is $\tau = \mu_{\text{val}} + 3\sigma_{\text{val}}$ on nominal validation scores; we also report $k \in \{1, 2, 3\}$, percentiles $\{95, 97.5, 99\}$, and a validation-oracle F1 threshold as a diagnostic only.

4 Experimental Evaluation

4.1 Corpus and Window Labels

We stream selective Thang6 day zips from a local campus extract (`D:/huflit-campus-logs`), skipping `fortigate.csv` and members >500 MiB. Proxy line labels use HTTP 4xx/5xx, deny/block fields, and keywords—not SOC adjudication. A window is positive iff *any* of its L lines is positive (OR aggregation). Under independence, $1 - (1 - p)^L$ with $p=0.0796$ predicts prevalence ≈ 0.81 , matching Protocol A’s 80.5% test rate.

4.2 Protocol A (leaky archive split)

Records are ordered by backup day $\rightarrow$ ZIP $\rightarrow$ member $\rightarrow$ in-member position, then split 80/10/10 before windowing. This is *not* event time: windows may cross file/service boundaries. A raw-line hash audit of this split found 36.235% of unique validation lines also in training (and 11.382% validation/test line overlap). Results: `huflit_baselines.json`.

4.3 Protocol B (dedup + source-bounded + day split)

We keep the first SHA-1 of each *raw line* (262,194 duplicates skipped while filling 1M unique lines), emit windows only inside one (day, source, member) group, and assign days 15–17 Jun $\rightarrow$ train, 18 Jun $\rightarrow$ val, 19 Jun $\rightarrow$ test. Day folders are backup collection periods, not verified per-event timestamps. Sources: `aca/careerhub/courses`. Test-window prevalence is 56.36% (always-anomaly F1 baseline 0.721).

Raw-line dedup does *not* imply unique template windows: different raw strings can map to the same template-ID sequence, and similar traffic on different days can yield identical windows. We therefore report **exact template-ID window sequence overlap** (SHA-1 of the integer window vector): 505 train/val and 110 train/test shared windows (`huflit_v2_split_meta.json`).

4.4 Experimental Setup

Table 2 lists corpus and model hyperparameters used for Protocol B (masked models). Protocol A uses the same architecture on the earlier stream-position arrays (`data/processed/huflit`). Training is nominal-only; seeds {42, 101, 202, 303, 404}; hardware is CPU (PyTorch 2.8.0+cpu, 2 intra-op threads).

4.5 Models and Metrics

We evaluate PCA count-vector reconstruction [6], a masked Transformer without TCN, and the masked TCN–Transformer. Protocol A also retains DeepLog/LogAnomaly-lite rows from the same-array suite. Metrics: F1 at $\mu+3\sigma$, ROC-AUC, AUPRC, **AUPRC lift over prevalence**, always-anomaly F1 baseline, and batch-1 PLL latency.

4.6 Results

On Protocol A, DeepLog leads ROC-AUC (0.781); the TCN is *worse* than no-TCN on mean F1/AUC and much slower at batch-1 PLL. The always-anomaly F1 baseline (0.892) exceeds every learned F1 at $\mu+3\sigma$; AUPRC must be read against prevalence 0.805.

On Protocol B, masked models lose day-to-day ranking ($\text{AUC} \approx 0.49\text{--}0.53$). PCA keeps a small positive AUPRC lift (+0.098). Table 5: $\mu+3\sigma$ starves recall;

Table 2. Protocol B setup (masked models). Window label = OR over L proxy line labels.

Item	Value
Unique lines kept / dups skipped	1,000,000 / 262,194
Train / val / test windows	59,966 / 19,988 / 19,990
Test positives / prevalence	11,267 / 56.36%
Always-anomaly F1 baseline (test)	0.721
Vocab (templates+pad+mask)	1,183
L , stride, mask ratio	20, 10, 15%
d_{model} , heads, layers, FFN	128, 4, 1, 256
TCN (when used)	causal conv $k=3$, dilations $\{1, 2\}$
Optimizer / LR / batch / epochs	Adam / 10^{-3} / 256 / 6
Dropout	0 (default encoder)
Scoring	leave-one-out PLL
Primary threshold	$\mu_{\text{val}} + 3\sigma_{\text{val}}$ on nominal val
Seeds	42, 101, 202, 303, 404
Hardware	CPU, 2 PyTorch threads

Table 3. Protocol A (leaky stream-position): $\tau = \mu + 3\sigma$; test prevalence 80.5%; always-anomaly F1 baseline ≈ 0.892 ; AUPRC no-skill ≈ 0.805

Model	F1	ROC-AUC	AUPRC	Lift	Batch-1 ms
Always-anomaly (trivial)	0.892	0.500	0.805	0.000	—
PCA	0.000 ± 0.000	0.738 ± 0.004	0.913 ± 0.003	+0.108	NR
DeepLog	0.011 ± 0.022	0.781 $\pm$ 0.020	0.882 ± 0.025	+0.077	NR
Masked Transformer (No TCN)	0.348 ± 0.410	0.769 ± 0.035	0.870 ± 0.036	+0.065	2.04
Masked TCN-Transformer	0.040 ± 0.016	0.723 ± 0.018	0.836 ± 0.014	+0.031	52.92

Table 4. Protocol B (dedup + source-bounded + day split): primary $\tau = \mu + 3\sigma$; test prevalence 56.4%; always-anomaly F1 baseline 0.721

Model	F1@ 3σ	ROC-AUC	AUPRC	Lift	Batch-1 ms
Always-anomaly (trivial)	0.721	0.500	0.564	0.000	—
PCA	0.000 ± 0.000	0.629 ± 0.008	0.661 ± 0.006	+0.098	—
Masked Transformer (No TCN)	0.001 ± 0.002	0.493 ± 0.009	0.511 ± 0.007	−0.052	18.54 ± 0.47
Masked TCN-Transformer	0.151 ± 0.183	0.532 ± 0.112	0.557 ± 0.124	−0.007	51.11 ± 9.08

Table 5. Protocol B threshold sensitivity for Masked TCN-Transformer (mean F1 over 5 seeds)

$\mu+1\sigma$	$\mu+2\sigma$	$\mu+3\sigma$	$p95$	$p99$	Oracle-val
0.696	0.585	0.151	0.300	0.181	0.741

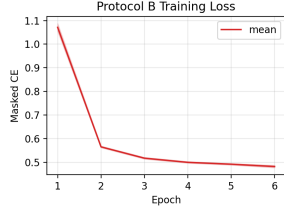


Fig. 2. Protocol B masked training loss (mean over seeds).

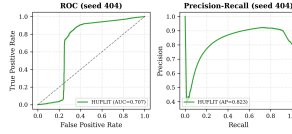


Fig. 3. Protocol A ROC/PR (leaky split; descriptive).

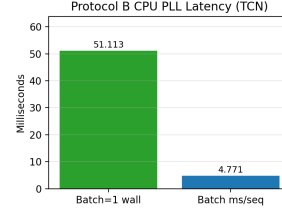


Fig. 4. Protocol B batch-1 vs batched PLL latency.

$\mu+1\sigma$ F1 (0.696) approaches the always-anomaly baseline (0.721) without improving AUC—loose thresholds harvest prevalence, not representation quality. Oracle-val F1 (0.741) is only marginally above the trivial baseline. TCN remains slower (51 vs. 19 ms) without consistent accuracy gains.

Due to privacy restrictions, raw campus telemetry cannot be redistributed. We retain code, configuration, aggregate split statistics, and derived evaluation JSON artifacts in the project repository for co-author reproduction; a public mirror is contingent on institutional approval.

5 Discussion & Threats to Validity

What sequence means: Protocol A orders archive members, not wall-clock events. Protocol B removes cross-member windows and duplicate raw lines but still uses backup-day folders rather than verified timestamps.

Window OR labels: OR aggregation manufactures high prevalence and makes the always-anomaly F1 baseline a necessary reference (not an upper bound: a perfect detector still has $F1=1$). Ranking metrics and lift-over-prevalence must accompany thresholded F1.

Proxy labels and label-feature coupling: HTTP status tokens, deny/block fields, and keyword heuristics both *define* proxy labels and often remain visible in template text. Models may therefore learn the labeling rule rather than operational anomalies. Gold SOC labels and keyword-ablated inputs are required before operational claims.

Ethics: Campus telemetry stays private; any future release needs redaction and authorization.

6 Conclusion

We showed that archive-order splits and OR window labels can inflate prevalence and make thresholded F1 look stronger than ranking warrants on HUFLIT campus logs. After deduplication, source-bounded windows, and day holdout, masked TCN/Transformer ranking collapses while PCA retains a modest AUPRC lift; TCN adds latency without a reliable benefit. Future work: event timestamps, SOC labels, Loghub [11] sanity checks under the same harness, keyword-ablated inputs, and—separately—measured local LLM narration.

Acknowledgment

The authors thank the Faculty of Information Technology, Ho Chi Minh City University of Foreign Languages - Information Technology (HUFLIT), Ho Chi Minh City, Vietnam for providing the production campus log telemetry and computing infrastructure. Generative AI tools assisted with bibliographic search, cross-checking, manuscript refinement, and English copy-editing; all citations were independently verified by the authors, and all core research, designs, implementations, and evaluations were conducted by the authors.

Disclosure of Interests

The authors have no competing interests to declare that are relevant to the content of this article.

References

1. He, P., Zhu, J., Zheng, Z., Lyu, M.R.: Drain: An Online Log Parsing Approach Based on Fixed Depth Tree. In: Proc. IEEE Int. Conf. Web Serv. (ICWS), pp. 33–40 (2017).
2. Du, M., Li, F., Zheng, G., Srikumar, V.: DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning. In: Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS), pp. 1285–1298 (2017).
3. Meng, W., Liu, Y., Zhu, Y., et al.: LogAnomaly: Unsupervised Detection of Sequential and Quantitative Anomalies in Unstructured Logs. In: Proc. 28th Int. Joint Conf. Artif. Intell. (IJCAI), pp. 4739–4745 (2019).
4. Zhang, X., Xu, Y., Lin, Q., et al.: Robust Log-Based Anomaly Detection on Unstable Log Data. In: Proc. 27th ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng. (ESEC/FSE), pp. 807–817 (2019).
5. Li, X., Chen, P., Jing, L., He, Z., Yu, G.: SwissLog: Robust Anomaly Detection and Localization for Interleaved Unstructured Logs. IEEE Trans. Dependable Secure Comput. 20(4), 2762–2780 (2023). <https://doi.org/10.1109/TDSC.2022.3162857>
6. Xu, W., Huang, L., Fox, A., Patterson, D., Jordan, M.I.: Mining Console Logs for Large-Scale System Problem Detection. In: Proc. 3rd Conf. SysML (2008).
7. Bai, S., Kolter, J.Z., Koltun, V.: An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling. arXiv preprint arXiv:1803.01271 (2018).
8. Vaswani, A., Shazeer, N., Parmar, N., et al.: Attention Is All You Need. In: Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 30, pp. 5998–6008 (2017).
9. Ahmed, T., Ghosh, S., Bansal, C., et al.: Recommending Root-Cause and Mitigation Steps for Cloud Incidents using Large Language Models. In: Proc. IEEE/ACM Int. Conf. Softw. Eng. (ICSE), pp. 1737–1749 (2023). <https://doi.org/10.1109/ICSE48619.2023.00149>
10. Soldani, J., Brogi, A.: Anomaly Detection and Failure Root Cause Analysis in (Micro)Service-Based Cloud Applications: A Survey. ACM Comput. Surv. 55(3), 1–39 (2022).
11. Zhu, J., He, S., He, P., Liu, J., Lyu, M.R.: Loghub: A Large Collection of System Log Datasets for AI-driven Log Analytics. In: Proc. 34th IEEE Int. Symp. Softw. Reliab. Eng. (ISSRE), pp. 355–366 (2023).
12. Du, M., Li, F.: Spell: Streaming Parsing of System Event Logs. In: Proc. IEEE Int. Conf. Data Min. (ICDM), pp. 859–864 (2016).
13. Ma, Z., Yang, J., Chen, T.H.: LLM4Log: A Systematic Review of Large Language Model-based Log Analysis. arXiv preprint arXiv:2604.16359 (2026).
14. Ma, L., Liu, J., Zhang, T., VanNostrand, P.M., Hofmann, D.M., Cao, L., Rundensteiner, E.A., Chen, J.: Krone: Hierarchical and Modular Log Anomaly Detection. In: Proc. 42nd IEEE Int. Conf. Data Eng. (ICDE), pp. 1773–1787 (2026).
15. Wu, Z., Wang, J., Zheng, L., Zhang, Y., Li, S., Chen, L.: Efficient Zero-shot and Label-free Log Anomaly Detection for Resource-constrained Systems (MaidLog). In: Proc. 42nd IEEE Int. Conf. Data Eng. (ICDE), pp. 671–684 (2026). <https://doi.org/10.1109/ICDE65706.2026.00056>
16. Liu, M., Wang, T., Zhao, Y., Xia, C., Zeng, Y., Tao, Y.: LogMUSE: Log Anomaly Detection via Multi-Scale Semantic Representation. IEEE Trans. Serv. Comput. (2026). <https://doi.org/10.1109/TSC.2026.3700403>
17. Dietz, S., Klede, K., Nguyen, A., Eskofier, B.M.: ContraLog: Log File Anomaly Detection with Contrastive Learning and Masked Language Modeling. arXiv preprint arXiv:2602.03678 (2026).
18. Pei, C., Cui, H., Li, J., et al.: Smart Eye: LLM-Guided Proposer-Verifier Framework for Industrial-Scale Log Anomaly Detection. In: Proc. ACM Web Conf. (WWW), pp. 7869–7879 (2026). <https://doi.org/10.1145/3774904.3792804>
19. Liao, N., Liu, Z.: Log Anomaly Detection Method Based on Transformer and Temporal Convolutional Networks. IEEE Access 13, 68547–68560 (2025). <https://doi.org/10.1109/ACCESS.2025.3561669>
20. Zhu, X., Tang, X., Wu, S., et al.: CoLA: Model Collaboration for Log-based Anomaly Detection. Proc. VLDB Endow. 18(11), 3979–3987 (2025). <https://doi.org/10.14778/3749646.3749668>